

Graph Neural Networks Powered by Encoder Embedding for Improved Node Learning

Shiyu Chen , Cencheng Shen , Youngser Park , and Carey E. Priebe , *Senior Member, IEEE*

Abstract—Graph neural networks (GNNs) have emerged as a powerful framework for a wide range of node-level graph learning tasks. However, their performance typically depends on random or minimally informed initial feature representations, where poor initialization can lead to slower convergence and increased training instability. In this paper, we address this limitation by leveraging a statistically grounded one-hot graph encoder embedding (GEE) as a high-quality, structure-aware initialization for node features. Integrating GEE into standard GNNs yields the GEE-powered GNN (GG) framework. Across extensive simulations and real-world benchmarks, GG provides consistent and substantial performance gains in both unsupervised and supervised settings. For node classification, we further introduce GG-C, which concatenates the outputs of GG and GEE and outperforms competing methods, achieving roughly 10–50% accuracy improvements across most datasets. These results demonstrate the importance of principled, structure-aware initialization for improving the efficiency, stability, and overall performance of graph neural network architecture, enabling models to better exploit graph topology from the outset.

Index Terms—Graph neural networks, graph encoder embedding, semi-supervised learning, unsupervised learning.

I. INTRODUCTION

GRAPHS are ubiquitous data structures that naturally capture relationships between objects, playing a crucial role in domains such as social networks, recommendation systems, and biological networks. Motivated by the pressing need to effectively learn from such graph-structured data, graph neural networks (GNNs) [1], [2], [3], [4], [5] have collected significant interests in recent years and delivered promising results on link prediction, node clustering, and node classification [6], [7], [8].

Despite much progress in GNN research [9], [10], [11], [12], the initialization of node embeddings remains a critical

yet underexplored aspect. Most existing GNNs rely either on raw node attributes or on simplistic initialization schemes such as degree-based features, random vectors, or identity matrices when explicit attributes are absent. These initializations provide limited structural signal. Degree features capture only local connectivity and cannot distinguish nodes with different global structural roles, while identity matrices treat each node as a distinct category and effectively force the model to learn structural relationships from scratch, often resulting in slow convergence and high sensitivity to hyperparameters. Random initialization provides no structural information at all. In contrast, structure-aware initializations such as spectral embeddings and Laplacian eigenvectors offer richer global information and are supported by strong theoretical guarantees under random graph models. However, they require costly eigendecomposition, whose computational overhead can exceed that of training the GNN itself, rendering these methods impractical for large-scale graphs.

To that end, the one-hot graph encoder embedding (GEE) [13] provides an appealing alternative that is both theoretically grounded and computationally efficient. Under standard random graph models, and given sufficiently many vertex labels, GEE is provably consistent: it converges to the latent vertex positions up to certain transformation, mirroring the theoretical guarantees of spectral embedding but without requiring any eigendecomposition. Instead of matrix factorization, GEE operates through a simple one-hot encoding of vertex labels followed by a single matrix multiplication, or equivalently, a single linear pass over the edgelist, enabling it to scale to graphs with millions of edges in seconds.

These properties make GEE a strong candidate for initializing GNNs. Prior work has demonstrated that GEE-generated embeddings achieve competitive or superior downstream performance relative to GNNs and spectral methods, while requiring only a fraction of the computational cost, across tasks such as node classification, clustering, multi-graph embedding, latent community detection, temporal graph analysis, and outlier detection [14], [15], [16], [17], [18]. These evidences highlight its potential as an effective feature initialization for enhancing GNN training stability, efficiency, and predictive performance.

While GEE performs competitively in many settings, it can sometimes fall short in capturing the intricate patterns and entangled inter-class relationships present in complex real-world networks. In contrast, neural network-based approaches excel at refining local patterns and modeling complex nonlinear structures, even when initialized with random or uninformative features. This contrast suggests a natural and effective

Received 15 July 2025; revised 26 January 2026; accepted 19 February 2026. Date of publication 23 February 2026; date of current version 5 June 2026. This work was supported by Microsoft Research. Recommended for acceptance by J. Wang. (*Corresponding author: Carey E. Priebe.*)

Shiyu Chen is with the Department of Applied Mathematics and Statistics (AMS), Johns Hopkins University, Baltimore, MD 21218 USA (e-mail: schen355@jh.edu).

Cencheng Shen is with the Microsoft Research, Redmond, WA 98052 USA (e-mail: c.shen@microsoft.com).

Youngser Park and Carey E. Priebe are with the Department of Applied Mathematics and Statistics (AMS), Center for Imaging Science (CIS), Mathematical Institute for Data Science (MINDS), Johns Hopkins University, Baltimore, MD 21218 USA (e-mail: youngser@jhu.edu; cep@jhu.edu).

This article has supplementary downloadable material available at <http://doi.org/10.1109/TPAMI.2026.3667397>, provided by the authors.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2026.3667397>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2026.3667397

integration: combining GEE's globally informed initialization with the GNN's local and nonlinear refinement may produce richer embeddings and ultimately yield superior downstream performance.

In this paper, we propose and evaluate the GEE-powered GNN (GG), which incorporates GEE-generated embedding as the initial node features within the standard Graph Convolutional Network (GCN) architecture [1], [2], applied to both clustering and classification tasks. In the node clustering task, GG demonstrates not only faster convergence but also consistently superior performance compared to the baseline GNN with randomized initialization, ranking first across all 16 real-world datasets evaluated. For node classification, we further develop a concatenated variant, GG-C, which demonstrates excellent performance across a wide range of training set sizes, from as low as 5% (essentially a semi-supervised setting) to 50% of the nodes used for training.

The remainder of the paper is organized as follows: Section II introduces our implementations of the GG method based on GNN and GEE. Section III provides an intuitive theoretical motivation for GG using embedding visualizations and convergence rate analyses. Section IV presents clustering performance results on both simulated and real-world datasets. Section V describes the classification adaptation of GG-C, which differs from the clustering setting due to label availability and changes in the learning objective. Section VI reports classification results under various training percentages on both synthetic and real data. Additional experiments are provided in the supplementary material. The source code of our method is available on Github.¹

II. CLUSTERING METHOD

A. GNN

The architecture considered in this paper utilizes the graph convolution network with residual skip connections [19] and employs the loss of deep modularity networks (DMoN) [8] to enhance community structure preservation.

1) *GCN*: The layer-wise propagation rule of GCN is defined as follows:

$$Z^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} Z^{(l)} W^{(l)} \right).$$

Here, $\tilde{A} = A + I_n$ is the adjacency matrix with added self-connections, where A is the original adjacency matrix, $I_n \in R^{n \times n}$ is the identity matrix and n is the number of nodes. The degree matrix $\tilde{D}$ of $\tilde{A}$ is defined as $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$. $W^{(l)}$ represents the trainable weight matrix for the l -th layer, and $\sigma(\cdot)$ denotes the activation function, which in this case is the ReLU function, defined as $\sigma(x) = \max(0, x)$. Finally, $Z^{(l)} \in R^{n \times d}$ is the activation matrix in the l -th layer, where $Z^{(0)}$ can be either randomized initial embeddings or specific node features. Here, d denotes the embedding dimension, which is set to match the number of clusters K .

2) *Skip Connections*: Skip connections add the initial features to the outputs of each GCN layer to obtain the final

representation $\hat{Z}$, which can be formulated as:

$$\hat{Z} = \sum_{l=0}^L Z^{(l)},$$

where L denotes the number of layers, by default 2. Skip connections are known to provide significant benefits in neural network architectures: they preserve features from earlier layers, mitigate vanishing gradients, and help reduce oversmoothing in GNNs.

3) *DMoN Loss*: The loss function of DMoN combines spectral modularity maximization with a regularization term to avoid trivial solutions and is formulated as:

$$C = \text{softmax}(\hat{Z}),$$

$$\mathcal{L}_{\text{DMoN}}(C; A) = -\frac{1}{2m} \text{Tr}(C^T B C) + \frac{\sqrt{K}}{n} \left\| \sum_i C_i^T \right\|_F - 1,$$

where $C \in R^{n \times K}$ is the soft community assignment matrix, obtained by applying the softmax function to the GNN output $\hat{Z}$. The adjacency matrix of the graph is denoted as A , and $m = \frac{1}{2} \sum_{i,j} A_{ij}$ is the total number of edges. The modularity matrix B is defined as: $B = A - \frac{vv^T}{2m}$, where v represents the degree vector of the graph, computed as: $v = A \mathbf{1}_n$, with $\mathbf{1}_n \in R^n$ being a column vector of ones.

The matrix trace $\text{Tr}(C^T B C)$ corresponds to the modularity maximization component, which encourages dense within-class connections while reducing between-class edges. The regularization term $\left\| \sum_i C_i^T \right\|_F$ computes the Frobenius norm of the aggregated community assignments, thereby preventing trivial solutions by mitigating the collapse of clusters.

4) *Prediction*: The soft community assignment matrix C , derived from the GNN output via a softmax function, serves as a probability matrix. Each node is assigned to the community with the highest probability:

$$\hat{Y} = \text{argmax}(C, \dim = 1).$$

To formalize the entire unsupervised GNN pipeline, we formulate it as a single function F_{GNN}^u that maps the initial features $Z^{(0)}$ and the graph structure A to the final label predictions $\hat{Y}$:

$$\hat{Y} = F_{\text{GNN}}^u(Z^{(0)}, A).$$

In the case of the vanilla GNN, we use a randomized initial embedding X_0 as the input of F_{GNN}^u :

$$\hat{Y}_{\text{GNN}} = F_{\text{GNN}}^u(X_0, A).$$

In our implementation, $X_0 \in R^{n \times k}$ is initialized using the Xavier uniform initialization, where each embedding element is sampled from $\mathcal{U}(-\sqrt{\frac{6}{k+n}}, \sqrt{\frac{6}{k+n}})$.

B. GEE

GEE is an efficient and flexible graph embedding method that can be applied with partial ground-truth labels, or labels generated by other methods, or without labels. The core step of supervised GEE is the multiplication of adjacency matrix A

¹<https://github.com/chenshy202/GG>

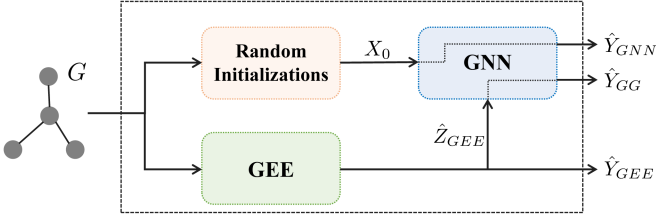


Fig. 1. A schematic diagram of the three algorithms for the node clustering task.

with a column-normalized one-hot encoding matrix W :

$$Z = AW, \quad W_{ik} = \frac{\mathbb{I}\{Y_i = k\}}{n_k}, \quad (1)$$

where $\mathbb{I}(\cdot)$ is the indicator function and $n_k = \sum_{j=1}^n \mathbb{I}\{Y_j = k\}$ is the size of community $k \in [1, K]$. In practice, graph data are typically stored in edgelist format, and this step can be efficiently performed with a single linear pass over the edgelist [13], [16], avoiding the need to construct an adjacency matrix.

Here, when node i belongs to community k , the entry W_{ik} equals $1/n_k$; Otherwise, it is 0. Consequently, each column of W sums up to 1 and can be interpreted as a probability distribution over the nodes in that community. Eventually, the i -th row Z_i summarizes how strongly node i is connected to all communities, with the k -th column $Z_{\cdot k}$ indicating connectivity to community k . An interesting property of GEE is that its embedding dimension naturally matches the number of communities K , eliminating the need to manually select an embedding dimension.

In the clustering task, we adopt the unsupervised version of GEE. The procedure starts with a random label vector, then repeats:

- 1) compute node embeddings with the supervised GEE, and
- 2) run k -means on the embeddings to update the labels.

The loop ends when the labels stop changing or when the maximum number of iterations M is reached. Formally, the whole routine can be viewed as a function $F_{GEE}^u(A, k, M)$:

$$(\hat{Z}_{GEE}, \hat{Y}_{GEE}) = F_{GEE}^u(A, k, M). \quad (2)$$

C. GEE-Powered GNN (GG)

We generate node features $\hat{Z}_{GEE}$ from (2), and utilize it as the input of the function F_{GNN}^u to obtain the predicted label vector $\hat{Y}_{GG}$:

$$\hat{Y}_{GG} = F_{GNN}^u(\hat{Z}_{GEE}, A).$$

Fig. 1 provides a schematic overview of the three algorithms, GNN, GEE, and GG, summarizing their workflows for the clustering task.

III. THEORETICAL MOTIVATION

Identifying meaningful community structure in graphs is a fundamental problem in network analysis [20], [21], [22], [23]. The goal is to uncover groups of vertices that exhibit stronger

intra-group connectivity than inter-group connectivity. Over the years, a wide range of approaches have been developed, including modularity-based methods [24], [25], spectral embedding [26], [27], likelihood-based approaches [28], [29], and random-walk-based methods [30], [31], among others.

Graph neural networks differ fundamentally from the above approaches in that they optimize an objective function via gradient descent, iteratively refining an initial set of node features [1], [2]. Intuitively, providing a stronger initialization can reduce the number of iterations required to reach a good solution, mitigate the risk of getting trapped in poor local minima, and potentially lead to improved downstream performance.

Recent work across diverse domains further supports the value of structurally-informed node representations. For instance, deep embedding methods such as SDNE [32], asymmetric transitivity preserving embeddings for directed graphs [33], relation-symmetry based contrastive learning for knowledge graphs [34], relational multi-view clustering [35], and robustness-oriented GNNs that downweight structurally uncertain neighbors [36] all demonstrate that explicitly encoding local, global, or relational structure can substantially improve embedding quality and downstream task performance.

This motivates our core proposal: a GEE-powered GNN, which remains a standard GNN architecture but is warm-started with node features generated by the statistically consistent GEE. By placing the initial embedding in a more informative and well-structured region of the feature space, GEE initialization potentially enables subsequent GNN training to refine representations more quickly and effectively than a vanilla GNN that relies on random or uninformative initializations [1].

To empirically validate this hypothesis, we visualize the evolution of node embeddings for both GG and the vanilla GNN during training on a DC-SBM graph. As illustrated in Fig. 2, the impact of our warm-starting strategy is immediately apparent. GG (top row) begins at epoch 0 with some discernible community structure (ARI = 0.05), visually confirming that it starts in a “promising region” of the embedding space. Then it rapidly refines this structure, converging in only 1,800 epochs to a high-quality partition with an ARI of 0.32. In comparison, the randomly initialized GNN (bottom row) starts from a completely unstructured state (ARI = 0.00). Its learning trajectory is substantially slower, showing negligible improvement even by Epoch 1000 (ARI = 0.01). Ultimately, it requires a much longer training process (8,180 epochs) to reach a slightly inferior solution (ARI = 0.26). This side-by-side comparison offers an empirical justification for our approach.

It is worth noting that, in principle, one could use any node embedding as an initialization for GNNs, including spectral embeddings, node2vec, or other representation-learning methods, since they all provide node features with certain theoretical guarantees. Intuitively, these warm starts should offer benefits similar to GEE; and theoretically, using any statistically consistent feature initialization ensures that the subsequent neural network refinement remains consistent as well under a proper random graph model.

However, the main limitation is computational cost: many of these embeddings require expensive matrix factorizations or

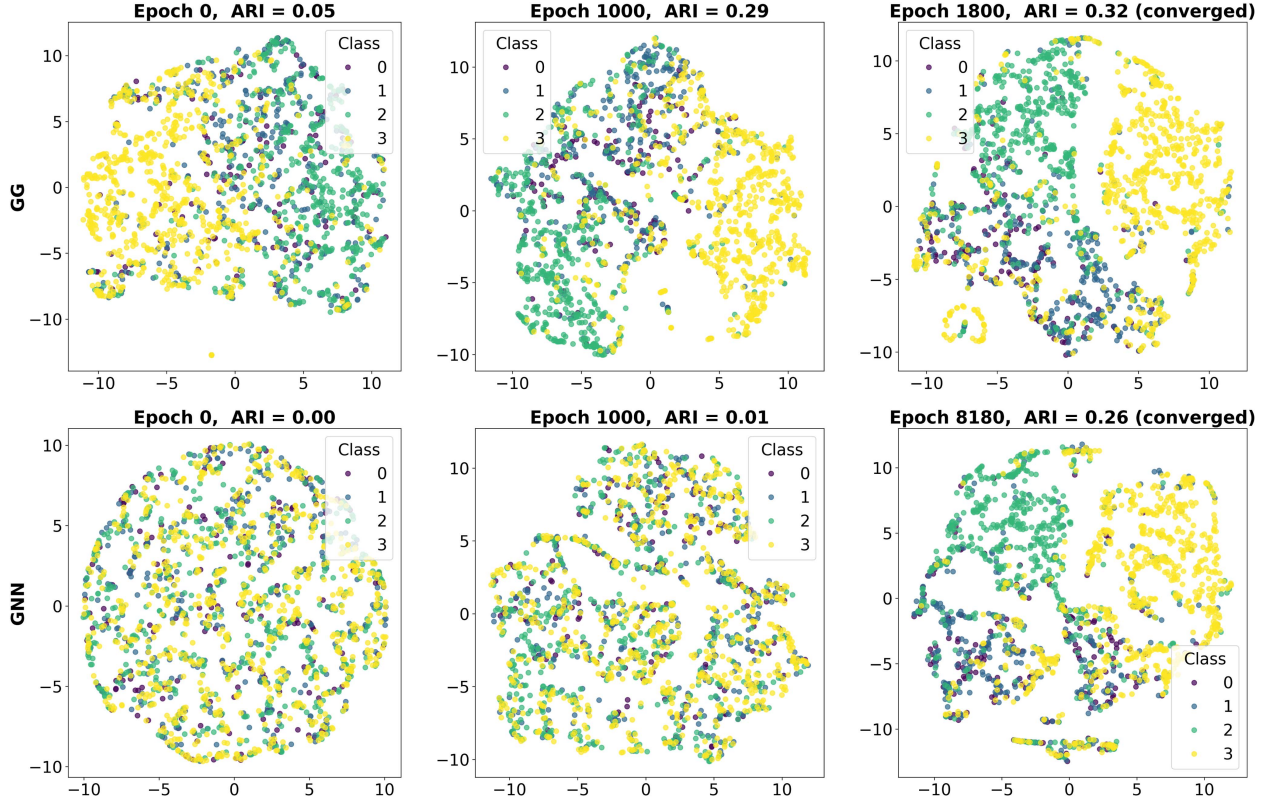


Fig. 2. Visualization of Training Dynamics and Clustering Quality. This figure contrasts the training process of our proposed GG (top row) and a vanilla GNN (bottom row) on a Degree-Corrected Stochastic Block Model (DC-SBM) graph (see Subsection IV-A for detailed settings). It demonstrates that GG’s warm-start leads to a better initial state, faster convergence, and a superior final clustering quality. GG begins with a structured embedding ($\text{ARI} = 0.05$) and converges in only 1,800 epochs to a high-quality partition ($\text{ARI} = 0.32$). In contrast, the GNN starts from a random state ($\text{ARI} \approx 0.00$) and requires a much longer training of 8,180 epochs to achieve an inferior result ($\text{ARI} = 0.26$).

optimization procedures already. In practice, it is often more efficient to simply train a GNN multiple times with random initializations and select the best outcome, than to compute a costly embedding solely for warm-starting. In contrast, GEE is both theoretically grounded and computationally lightweight, making it a practical initialization method for large-scale graphs.

IV. CLUSTERING EVALUATION

To systematically evaluate the performance of the proposed GG method, we conduct experiments on both synthetic and real-world datasets. The evaluation considers two parts: clustering accuracy, where we compare GG against GEE and the vanilla GNN; and running time, focusing on the convergence time difference between GG and the vanilla GNN.

For each graph, whether real-world or synthetic, we generate three distinct sets of label predictions: $\hat{Y}_{GEE}$ from GEE, $\hat{Y}_{GNN}$ from a vanilla GNN, and $\hat{Y}_{GG}$ from the proposed GG. We then assess the quality of these predictions against the ground-truth labels using the Adjusted Rand Index (ARI). The ARI measures the similarity between two clusterings, where a score of 1 indicates a perfect match, and values near 0 suggest random partitioning.

A. Simulation

For simulations, we consider the stochastic block model (SBM) [37] and the degree-corrected SBM (DC-SBM) [38].

Here we present the results on DC-SBM, as it better captures the degree heterogeneity in real-world networks. Results for SBM are available in Appendix, available online.

In DC-SBM, the generative process for an edge between two nodes depends on three components: their community assignments, a shared block probability matrix, and individual degree parameters. First, each node i is assigned to a community, represented by a label $Y_i \in \{1, \dots, K\}$. The connectivity between these communities is governed by the block probability matrix $B \in [0, 1]^{K \times K}$, where entry $B(k, l)$ specifies the probability of a link between a node in community k and a node in community l . The key distinction from the standard SBM is the introduction of a degree parameter θ_i for each node to account for degree heterogeneity.

Combining these elements, the adjacency matrix $A \in \{0, 1\}^{n \times n}$ is generated as follows:

$$A_{ij} \sim \text{Bernoulli}(\theta_i \theta_j B(Y_i, Y_j))$$

where $A(i, i) = 0$ and $A(j, i) = A(i, j)$ to ensure the graph is simple and undirected.

For the experimental setup, the graphs are configured with $n = 2000$ nodes and $K = 4$ communities. To evaluate model performance under varying degrees of community separability, we fix the intra-community edge probability at 0.3 while systematically varying the inter-community probability r from 0 to 0.3 in increments of 0.02. To introduce degree heterogeneity, the degree correction parameters θ_i are drawn from a Beta

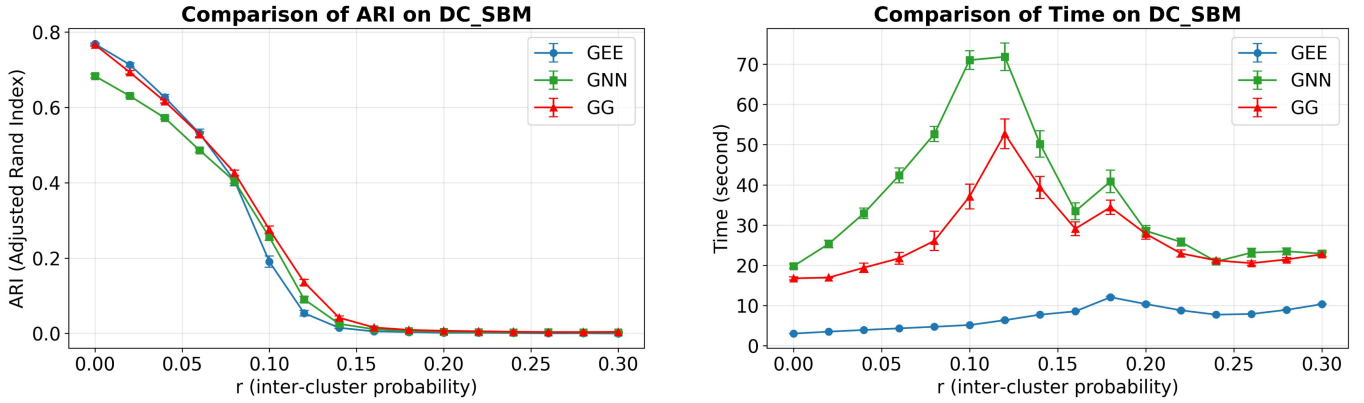


Fig. 3. Clustering performance comparison of GEE, GNN and GG on DC-SBM graphs. This figure shows that our proposed GG offers the best trade-off between clustering accuracy (ARI, left) and speed (Time, right). While GEE is fastest, GG matches or surpasses its accuracy, especially on more challenging graphs ($r > 0.08$). Compared to the vanilla GNN, GG is both more accurate and substantially faster across all conditions.

distribution, $\beta(1, 4)$. Furthermore, we impose a structural imbalance by setting the community sizes to a ratio of 1:2:3:4, a choice intended to better reflect the skewed community size distributions often found in real-world networks.

To ensure the statistical reliability of our findings across all simulation experiments (including SBM in Appendix, available online), we conduct 50 independent runs for each graph generated at every value of r for all simulations. We then report the mean and standard error of the performance and running time, which are visualized as error bars in the result plots. The visualizations of node embeddings presented in Fig. 2 are taken from a representative run of this experimental setup, specifically using a graph generated with $r = 0.1$.

Fig. 3 summarizes the performance. In terms of clustering accuracy (left panel), the results reveal the distinct roles of each component. As anticipated, GEE demonstrates high accuracy when the community structure is clear (low r), which degrades as the network becomes more complex ($r > 0.1$). The vanilla GNN, while performing more consistently than GEE in this regime, yields suboptimal results. This is the exact scenario where GG is designed to solve, which consistently outperforms the vanilla GNN across the entire difficulty spectrum. This demonstrates that GEE’s high-quality initialization allows the GNN component to avoid poor local optima and fine-tune its way to a superior solution. Regarding computational time (right panel), GEE is the fastest method, the vanilla GNN is the most costly, while GG strikes a fine balance, costing less than vanilla due to the better initialization.

B. Real Data

We evaluate the performance of the proposed method on a diverse suite of 16 publicly available real-world graphs. Following the methodology of prior work [13], [39], [40], we select datasets from public sources². This collection includes benchmark citation networks (e.g., Cora, Citeseer), co-authorship graphs (e.g.,

TABLE I
SUMMARY OF DATASETS USED IN EXPERIMENTS

Dataset	Vertices	Edges	Classes	Community sizes
Cora	2708	5429	7	818, 426, 418, 351
Citeseer	3327	4732	6	701, 668, 596, 590
ACM	3025	13128	3	1061, 999, 965
Chameleon	2277	18050	5	521, 460, 456, 453
DBLP	4057	3528	4	1197, 1109, 1006, 745
EAT	399	203	4	102, 99, 99, 99
UAT	1190	13599	4	299, 297, 297, 297
BAT	131	1074	4	35, 32, 32, 32
KarateClub	34	156	4	13, 12, 5, 4
Email	1005	25571	42	109, 92, 65, 61
Gene	1103	1672	2	613, 490
Industry	219	630	3	133, 44, 42
LastFM	7624	27806	17	1303, 1098, 655, 515
PolBlogs	1490	33433	2	636, 588
Wiki	2405	16523	17	406, 361, 269, 228
TerroristRel	881	8592	3	633, 181, 67

ACM, DBLP), social and communication networks (e.g., PolBlogs, LastFM, Email), web graphs based on hyperlinks (e.g., Chameleon, Wiki), biological interaction networks (e.g., Gene) and others representing a variety of structural properties and community distribution patterns.

The key statistics of these datasets are summarized in Table I. The table also details the size of each community within the datasets. For datasets with more than four communities, the sizes of the four largest communities are listed. As shown in the table, imbalanced community sizes are a common characteristic of these networks. To ensure reliable results, all experiments are repeated for 50 independent trials, and we report the mean and standard error for both ARI and running time. The detailed results are presented in Tables II, with the best ARI score for each

²Available: <https://github.com/cshen6/GraphEmd> and <https://github.com/yueliu1999/Awesome-Deep-Graph-Clustering>

TABLE II
CLUSTERING ACCURACY (ARI) AND RUNNING TIME ON REAL-WORLD DATASETS. THE BEST RESULT IS HIGHLIGHTED IN **BOLD**. RUNNING TIME FOR GEE IS EXCLUDED FROM COMPARISON WITH GNN-BASED MODELS DUE TO ITS NON-ITERATIVE NATURE.

	UAT	Industry	Wiki	Cora	LastFM	Email	KarateClub	PolBlogs
ARI								
GEE	0.090±0.001	0.061±0.006	0.116±0.002	0.137±0.004	0.341±0.004	0.436±0.005	0.449±0.014	0.812±0.001
GNN	0.126±0.002	0.099±0.005	0.106±0.003	0.111±0.004	0.274±0.009	0.340±0.003	0.725±0.011	0.745±0.003
GG	0.140±0.003	0.126±0.005	0.124±0.002	0.166±0.004	0.409±0.004	0.537±0.002	0.726±0.013	0.821±0.003
Running time (s)								
GEE	5.9±0.0	0.6±0.0	7.1±0.0	3.4±0.0	14.2±0.1	8.3±0.03	0.3±0.0	6.6±0.1
GNN	12.0±0.2	2.8±0.1	104.6±5.1	80.6±2.5	779.6±48.1	29.4±0.4	1.8±0.0	17.5±1.0
GG	12.4±0.3	2.9±0.1	65.1±3.7	59.2±3.7	474.1±29.9	28.2±0.3	1.8±0.0	10.4±0.1
	Gene	TerroristRel	DBLP	EAT	Citeseer	ACM	Chameleon	BAT
ARI								
GEE	0.008±0.002	0.062±0.008	0.018±0.001	0.017±0.001	0.046±0.003	0.033±0.003	0.045±0.002	0.043±0.003
GNN	0.008±0.002	0.075±0.005	0.009±0.001	0.023±0.002	0.035±0.002	0.034±0.003	0.056±0.002	0.060±0.003
GG	0.011±0.002	0.078±0.009	0.022±0.001	0.044±0.005	0.056±0.003	0.059±0.003	0.060±0.002	0.085±0.007
Running time (s)								
GEE	1.3±0.0	2.5±0.1	2.6±0.0	2.6±0.0	3.2±0.0	5.8±0.0	13.0±0.1	0.7±0.0
GNN	8.4±0.3	7.9±0.2	85.9±5.2	4.6±0.1	93.7±3.7	54.0±2.9	44.6±2.1	2.1±0.0
GG	7.9±0.1	7.8±0.2	70.3±3.2	4.5±0.1	63.5±3.0	47.8±2.8	31.3±1.0	2.1±0.0

dataset highlighted in bold. The results clearly demonstrate the advantages of GG. Not only does it achieve the best accuracy on all real-world datasets, but it also operates with substantially less computational time than the vanilla GNN in most cases. This empirical observation aligns closely with the simulation results.

V. CLASSIFICATION METHOD

A. GNN

For node classification, we employ the same graph convolution network architecture except a different loss function. The layer-wise update follows as:

$$Z^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} Z^{(l)} W^{(l)} \right),$$

$$\hat{Z} = \sum_{l=0}^L Z^{(l)}.$$

The model is trained by minimizing the cross-entropy loss $\mathcal{L}_{CE}$, over the set of training nodes $\mathcal{V}_{\text{train}}$. Given the output $\hat{Z}$, and ground-truth labels Y , the loss is defined as:

$$\mathcal{L}_{CE} = -\frac{1}{|\mathcal{V}_{\text{train}}|} \sum_{i \in \mathcal{V}_{\text{train}}} \log \left(\frac{\exp(\hat{Z}_{iY_i})}{\sum_{c=1}^K \exp(\hat{Z}_{ic})} \right). \quad (3)$$

Once trained, the model produces the final prediction $\hat{Y}$. The predicted class label for each node i is then determined by applying the argmax operator:

$$\hat{Y}_i = \arg \max_c (\hat{Z}_{ic}), \quad c \in [0, K-1].$$

To represent this entire end-to-end process, from the initial node representations $Z^{(0)}$ and the adjacency matrix A to the

final label predictions, we define a single function F_{GNN}^a . The output is thus expressed as:

$$\hat{Y} = F_{GNN}^a(Z^{(0)}, A).$$

For the vanilla GNN baseline, we use the same random initialization scheme as in the clustering task, generating an initial embedding matrix X_0 as the input of F_{GNN}^u :

$$\hat{Y}_{GNN} = F_{GNN}^a(X_0, A).$$

B. GEE

For GEE in the classification task, we generate node embeddings $\hat{Z}_{GEE}$ using the supervised GEE algorithm (1), where the labels for test nodes $\mathcal{V}_{\text{test}}$ are masked (e.g., set to -1) and not used by the GEE algorithm. We denote the masked label vector $\tilde{Y}$ as:

$$\tilde{Y}_i = \begin{cases} Y_i & \text{if } i \in \mathcal{V}_{\text{train}} \\ -1 & \text{if } i \in \mathcal{V}_{\text{test}} \end{cases} \quad (4)$$

Using this masked label vector, the supervised GEE embeddings are then computed as:

$$\hat{Z}_{GEE} = AW, \quad W_{ik} = \frac{\mathbb{I}\{\tilde{Y}_i = k\}}{n_k}. \quad (5)$$

To output labels for the test nodes, we use the linear discriminant analysis (LDA) on $\hat{Z}_{GEE}$:

$$\hat{Y}_{GEE} = \text{LDA}(\hat{Z}_{GEE}). \quad (6)$$

C. GG

Following the same warm-start strategy in the clustering task, the GG method for classification initializes GNN with features derived from GEE. Specifically, we utilize the supervised GEE

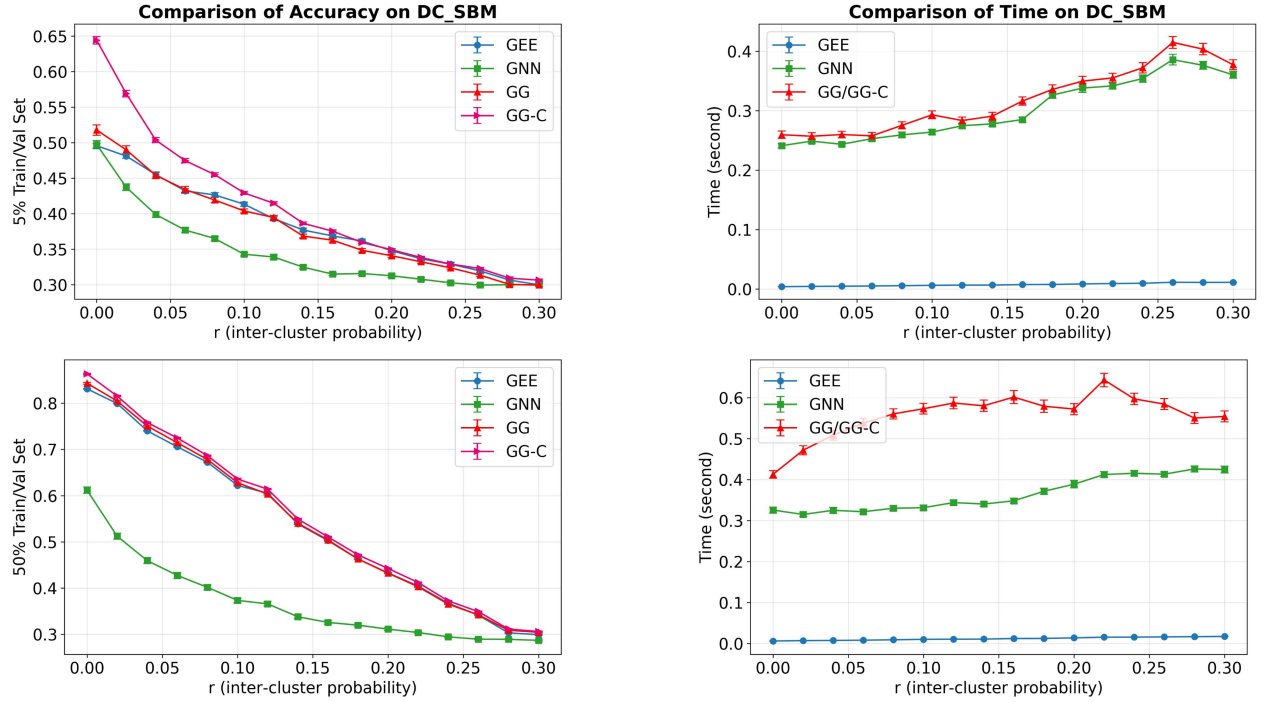


Fig. 4. Classification performance comparison of GEE, GNN, GG and GG-C on DC-SBM graphs with 5% (top row) and 50% (bottom row) train/val set. This figure demonstrates the superiority of our GG and GG-C. They overcome the poor accuracy of the faster GNN (which is prone to local optima), while our concatenated variant, GG-C, also surpasses the classic GEE, particularly in the challenging low-label (5%) setting.

embeddings, $\hat{Z}_{GEE}$ (from (5)) as input of the function F_{GNN}^a to obtain the predicted label vector $\hat{Y}_{GG}'$:

$$\hat{Y}_{GG} = F_{GNN}^a(\hat{Z}_{GEE}, A).$$

D. GG-C

Recognizing that GEE and GG may capture complementary aspects of the graph, where GEE emphasizing global structure and GG refining local patterns, we introduce a variant termed GG-C, which concatenates the respective embeddings to form an even richer representation for each node. Specifically, we concatenate the output of the supervised GEE, denoted as $\hat{Z}_{GEE}$, with the final GNN-refined embeddings from the GG model, denoted as $\hat{Z}_{GG}$. The concatenated embedding is then passed to an LDA classifier for final label prediction. Formally, the prediction can be expressed as:

$$\hat{Y}_{GG-C} = \text{LDA}([\hat{Z}_{GG}, \hat{Z}_{GEE}]).$$

This design is particularly advantageous for node classification. If either embedding component contains little discriminative information, the LDA classifier remains unaffected; if one or both components offer additional informative signals, whether individually or jointly, LDA is well-positioned to extract and leverage them. This idea is conceptually similar to the synergistic effect of multiple graph representations using GEE [17].

VI. CLASSIFICATION EVALUATION

In this section, we evaluate the node classification performance of the proposed methods. The evaluation is conducted

on both synthetic graphs generated by the SBM and DC-SBM, as well as on the same set of real-world datasets used in the clustering analysis. A key part of this evaluation is to assess model performance under varying levels of data scarcity, which is achieved through different training sizes.

A. Experimental Setup

We systematically vary the proportion of labeled nodes available for training by four different training sizes. Specifically, we set the ratio of the test set ($\mathcal{V}_{\text{test}}$) to the combined training and validation sets (train/val set) to be 19:1, 9:1, 4:1, and 1:1. This corresponds to the use of 5%, 10%, 20%, and 50% of the nodes for training and validation, respectively. Furthermore, to ensure that every class is represented during training and validation, we enforce a minimum of two training nodes and one validation node per class.

From this combined training and validation pool, we further allocate 10% of the nodes for the validation set ($\mathcal{V}_{\text{val}}$) and the remaining 90% for the training set ($\mathcal{V}_{\text{train}}$). Each method is evaluated by the classification accuracy. For a given set of nodes $\mathcal{V}$, accuracy is defined as the fraction of correctly classified nodes. We denote this metric as $\text{Acc}(\mathcal{V})$ and formalize it as:

$$\text{Acc}(\mathcal{V}) = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \mathbb{I}(\hat{Y}_i = Y_i),$$

where $\hat{Y}_i$ is the predicted label for node i and Y_i is its ground-truth label.

During training, we employ an early stopping strategy for the GNN-based methods by monitoring accuracy on a separate

TABLE III

CLASSIFICATION ACCURACY AND RUNNING TIME ON REAL-WORLD DATASETS OF 5% (TOP) AND 50% (BOTTOM) TRAIN/VAL DATA. FOR ACCURACY, THE BEST AND SECOND-BEST RESULTS ARE HIGHLIGHTED IN **BOLD** AND *ITALIC*, RESPECTIVELY. FOR THE GNN-BASED MODELS (GNN, GG AND GG-C), THE FASTEST RUNNING TIME IS ALSO MARKED IN BOLD. THE RUNNING TIME OF GEE IS NOT HIGHLIGHTED AND IS EXCLUDED FROM DIRECT COMPARISON DUE TO ITS NON-ITERATIVE NATURE.

5%	ACM	BAT	DBLP	EAT	UAT	Wiki	Gene	IIP
Accuracy								
GEE	0.469±0.002	<i>0.412±0.006</i>	0.316±0.001	<i>0.364±0.005</i>	<i>0.415±0.005</i>	0.244±0.004	0.575±0.004	0.410±0.014
GNN	0.427±0.004	0.294±0.005	0.297±0.002	0.321±0.005	0.358±0.005	0.224±0.003	0.525±0.006	0.515±0.008
GG	<i>0.492±0.005</i>	0.382±0.007	<i>0.329±0.002</i>	0.351±0.005	0.404±0.006	0.326±0.003	0.531±0.005	<i>0.510±0.010</i>
GG-C	0.534±0.003	0.425±0.008	0.336±0.003	0.370±0.005	0.435±0.004	<i>0.304±0.003</i>	<i>0.549±0.003</i>	0.444±0.010
Running time (s)								
GEE	0.01±0.00	0.00±0.00	0.00±0.00	0.00±0.00	0.01±0.00	0.01±0.00	0.00±0.00	0.00±0.00
GNN	0.37±0.01	0.14±0.00	0.30±0.01	0.21±0.01	0.32±0.01	0.74±0.02	0.16±0.00	0.13±0.00
GG/GG-C	0.37±0.01	0.14±0.00	0.28±0.01	0.22±0.01	0.30±0.01	0.77±0.02	0.14±0.00	0.13±0.00

5%	LastFM	PolBlogs	TerroristRel	KarateClub	Chameleon	Cora	Citeseer
Accuracy							
GEE	0.333±0.004	0.723±0.005	<i>0.737±0.010</i>	0.648±0.016	0.267±0.004	0.349±0.006	0.244±0.002
GNN	0.431±0.003	0.706±0.009	0.683±0.006	0.257±0.012	<i>0.288±0.003</i>	0.318±0.006	0.238±0.003
GG	0.536±0.003	<i>0.729±0.014</i>	0.718±0.011	<i>0.663±0.019</i>	0.270±0.004	<i>0.393±0.005</i>	<i>0.280±0.003</i>
GG-C	<i>0.509±0.003</i>	0.815±0.009	0.765±0.006	0.729±0.014	0.333±0.003	0.426±0.004	0.303±0.003
Running time (s)							
GEE	0.01±0.00	0.01±0.00	0.00±0.00	0.00±0.00	0.01±0.00	0.00±0.00	0.00±0.00
GNN	1.58±0.04	0.34±0.01	0.26±0.01	0.13±0.00	0.57±0.01	0.38±0.01	0.36±0.01
GG/GG-C	2.18±0.05	0.30±0.01	0.25±0.01	0.13±0.00	0.56±0.02	0.39±0.02	0.38±0.01

50%	ACM	BAT	DBLP	EAT	UAT	Wiki	Gene	Industry
Accuracy								
GEE	0.749±0.002	0.350±0.008	0.538±0.001	0.314±0.008	0.497±0.003	0.592±0.001	0.762±0.002	0.601±0.005
GNN	0.544±0.008	<i>0.370±0.007</i>	0.379±0.005	<i>0.357±0.005</i>	0.385±0.006	0.406±0.006	0.606±0.009	<i>0.511±0.006</i>
GG	<i>0.751±0.004</i>	0.319±0.008	<i>0.532±0.002</i>	0.248±0.005	0.439±0.005	0.617±0.001	0.704±0.009	<i>0.614±0.005</i>
GG-C	0.759±0.002	0.394±0.007	0.536±0.002	0.385±0.007	<i>0.472±0.004</i>	<i>0.612±0.001</i>	<i>0.744±0.003</i>	0.624±0.005
Running time (s)								
GEE	0.01±0.00	0.00±0.00	0.00±0.00	0.00±0.00	0.01±0.00	0.01±0.00	0.00±0.00	0.00±0.00
GNN	0.40±0.01	0.14±0.00	0.32±0.01	0.24±0.01	0.37±0.01	0.85±0.04	0.19±0.01	0.14±0.00
GG/GG-C	0.70±0.03	0.13±0.00	0.54±0.02	0.21±0.01	0.47±0.02	0.93±0.02	0.18±0.01	0.14±0.00

50%	LastFM	PolBlogs	TerroristRel	KarateClub	Chameleon	Cora	Citeseer
Accuracy							
GEE	0.717±0.001	<i>0.907±0.002</i>	<i>0.899±0.001</i>	<i>0.826±0.011</i>	0.263±0.004	0.724±0.001	0.545±0.001
GNN	0.621±0.003	0.737±0.012	0.712±0.012	0.390±0.015	<i>0.281±0.002</i>	0.477±0.006	0.373±0.005
GG	0.773±0.001	0.869±0.011	0.865±0.010	0.803±0.014	0.320±0.004	<i>0.739±0.002</i>	<i>0.577±0.002</i>
GG-C	<i>0.766±0.001</i>	0.915±0.002	0.908±0.002	0.838±0.009	0.379±0.004	0.749±0.002	0.582±0.001
Running time (s)							
GEE	0.02±0.00	0.01±0.00	0.01±0.00	0.00±0.00	0.02±0.00	0.00±0.00	0.00±0.00
GNN	1.58±0.04	0.43±0.01	0.29±0.01	0.12±0.00	0.62±0.03	0.46±0.02	0.44±0.01
GG/GG-C	2.80±0.08	0.48±0.03	0.36±0.02	0.12±0.00	1.05±0.05	0.78±0.03	0.74±0.02

validation set, $\text{Acc}(\mathcal{V}_{\text{val}})$. The model parameters from the epoch that yields the highest validation accuracy are restored for the final evaluation on the test set, $\text{Acc}(\mathcal{V}_{\text{test}})$. To ensure a fair comparison, GEE is also evaluated under the same conditions, where the labels of nodes in the validation set are masked alongside the test set labels and are not used during its embedding generation process.

Regarding the running time of GG-C, it is reported jointly with GG in our figures and tables. This is because the additional computation in GG-C, consisting only of a simple feature concatenation and an LDA step, is negligible compared to the

shared GNN training process. As a result, the running times of GG and GG-C are almost always identical.

B. Simulation

Here we present two most representative train/val set proportions on DC-SBM: 50% (data-rich) and 5% (data-scarce), in Fig. 4. The complete results, including those for the 10% and 20% train/val set proportions on DC-SBM and all results for SBM, are provided in Appendix, available online. The graph generation parameters for this classification task are identical to

those used in the clustering evaluation, except that we perform 200 independent runs.

From the left two panels, it is evident that both GG and GG-C significantly outperform the vanilla GNN. When provided with sufficient training data (e.g., 50% of nodes labeled), GEE performs remarkably well, aligning with its theoretical guarantees. In such cases, GG and GG-C achieve similar high accuracy, indicating that GEE alone is already near-optimal. As the proportion of training/validation data decreases (see Appendix (available online) for results under 10% and 20% training), GEE's advantage decreases, though it still remains superior to the vanilla GNN.

In the semi-supervised regime, GG provides meaningful refinement over GEE, but its standalone accuracy does not substantially exceed GEE's until GG-C is introduced. This reveals an important insight: GEE supplies a strong global initialization, GG contributes nontrivial local refinement, and their concatenation yields a fused representation that effectively synergizes GEE's global structural information with the GNN's localized, task-specific adjustments.

From the right two panels, an interesting observation emerges: the vanilla GNN appears to converge slightly faster than the GG-based models. However, this should not be interpreted as better efficiency. Rather, it reflects early termination to a suboptimal local minimum. Note that the time difference between GG and vanilla GNN is marginal and negligible across all classification tasks.

C. Real Data

For experiments on real-world datasets, we performed 100 independent runs for each data split to ensure statistical robustness. All publicly available datasets were included in our evaluation, with the exception of the Email dataset, which was excluded due to several classes containing fewer than three samples, violating our minimum class size requirement. Results for the 5% and 50% training/validation set proportions are presented in Table III. Additional results for the 10% and 20% settings are provided in Appendix, available online.

Both GG and GEE serve as strong baselines, and in most cases outperform the vanilla GNN. Neither consistently dominates the other across all datasets. By contrast, the GG-C version, which fuses the embeddings from both GG and GEE, successfully captures complementary information from both global and local representations. As a result, it achieves superior performance across a diverse set of graph data. In nearly every case, GG-C either ranks as the top performer or closely follows the best-performing method in terms of classification accuracy. Finally, an expanded set of evaluations is provided in appendix (available online), including a comparison against the recent CutSSL [41] baseline and an analysis of our models' performance with increased layer depth.

VII. CONCLUSION

In this work, we introduced the GEE-powered GNN framework, motivated by the hypothesis that informative node features can substantially improve upon random initializations in graph

neural networks, alleviating issues such as slow convergence and suboptimal solutions. Through extensive experiments on both synthetic and real-world datasets, we demonstrated that the proposed method consistently enhances performance across node-level tasks. For node clustering, the GEE-based warm-start leads to faster convergence and superior clustering quality. For node classification, our concatenated variant achieves excellent performance by retaining the global information captured by GEE while benefiting from the local refinements introduced by the GNN.

These findings point to a promising direction in GNN research: rather than focusing solely on deeper or more sophisticated neural architectures, substantial improvements can be achieved by adopting high-quality feature initialization. Such strategies act as a form of implicit regularization, improving generalization and reducing the risk of overfitting, particularly in low-label or semi-supervised settings. Among the many available graph embedding algorithms, we chose GEE for its statistical consistency and computational scalability, but other initialization strategies could certainly be explored in future work.

The broader implications of this work extend beyond graph learning. The idea of leveraging statistically informed, structure-aware initializations is applicable to a wide range of neural network architectures, both for graph-structured and conventional data. Several promising directions remain, including exploring alternative warm-start strategies and deeper architectures; extending the approach to dynamic or temporal graphs, where initializations can adapt to evolving structure; and investigating applications in domains such as recommendation systems or computer vision, where combining pretrained global features with local refinement may offer similar advantages.

ACKNOWLEDGMENT

The authors thank the editor and anonymous reviewers for their valuable feedback.

REFERENCES

- [1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [2] T. N. Kipf and M. Welling, "Variational graph auto-encoders," 2016, *arXiv:1611.07308*.
- [3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [5] M. Schlichtkrull et al., "Modeling relational data with graph convolutional networks," in *Proc. 15th Int. Conf. Semantic Web*, Heraklion, Crete, Greece, 2018, pp. 593–607.
- [6] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 5161–5170.
- [7] M. Zhang, P. Li, Y. Xia, K. Wang, and L. Jin, "Labeling trick: A theory of using graph neural networks for multi-node representation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 9061–9073.
- [8] A. Tsitsulin, J. Palowitch, B. Perozzi, and E. Müller, "Graph clustering with graph neural networks," *J. Mach. Learn. Res.*, vol. 24, no. 127, pp. 1–21, 2023.
- [9] H. Cai, V. W. Zheng, and K. C.-C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018.

- [10] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [11] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 249–270, Jan. 2022.
- [12] C. E. Priebe, C. Shen, N. Huang, and T. Chen, "A simple spectral failure mode for graph convolutional networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 8689–8693, Nov. 2022.
- [13] C. Shen, Q. Wang, and C. E. Priebe, "One-hot graph encoder embedding," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 7933–7938, Jun. 2023.
- [14] C. Shen, J. Larson, H. Trinh, and C. E. Priebe, "Refined graph encoder embedding via self-training and latent community recovery," *Social Netw. Anal. Mining*, vol. 15, 2025, Art. no. 14.
- [15] C. Shen, Y. Dong, C. E. Priebe, J. Larson, H. Trinh, and Y. Park, "Principal graph encoder embedding and principal community detection," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 424–437, 2026.
- [16] C. Shen, J. Larson, H. Trinh, X. Qin, Y. Park, and C. E. Priebe, "Discovering communication pattern shifts in large-scale labeled networks using encoder embedding and vertex dynamics," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 2100–2109, Mar./Apr. 2024.
- [17] C. Shen, C. E. Priebe, J. Larson, and H. Trinh, "Synergistic graph fusion via encoder embedding," *Inf. Sci.*, vol. 678, 2024, Art. no. 120912.
- [18] C. Shen, "Encoder embedding for general graph and node classification," *Appl. Netw. Sci.*, vol. 9, 2024, Art. no. 66.
- [19] G. Li, M. Muller, A. Thabet, and B. Ghanem, "DeepGCNs: Can GCNs go as deep as CNNs?," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 9266–9275.
- [20] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," *Proc. Nat. Acad. Sci. USA*, vol. 99, no. 12, pp. 7821–7826, 2002.
- [21] M. E. J. Newman, "Detecting community structure in networks," *Eur. Phys. J. B*, vol. 38, no. 2, pp. 321–330, 2004.
- [22] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, no. 3/5, pp. 75–174, 2010.
- [23] B. Karrer and M. E. J. Newman, "Stochastic blockmodels and community structure in networks," *Phys. Rev. E*, vol. 83, 2011, Art. no. 016107.
- [24] V. D. Blondel, J. L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mechanics: Theory Experiment*, vol. 10008, 2008, Art. no. 6.
- [25] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," *Sci. Rep.*, vol. 9, 2019, Art. no. 5233.
- [26] K. Rohe, S. Chatterjee, and B. Yu, "Spectral clustering and the high-dimensional stochastic blockmodel," *Ann. Statist.*, vol. 39, no. 4, pp. 1878–1915, 2011.
- [27] D. Sussman, M. Tang, D. Fishkind, and C. Priebe, "A consistent adjacency spectral embedding for stochastic blockmodel graphs," *J. Amer. Stat. Assoc.*, vol. 107, no. 499, pp. 1119–1128, 2012.
- [28] C. Gao, Z. Ma, A. Y. Zhang, and H. H. Zhou, "Community detection in degree-corrected block models," *Ann. Statist.*, vol. 46, no. 5, pp. 2153–2185, 2018.
- [29] E. Abbe, "Community detection and stochastic block models: Recent developments," *J. Mach. Learn. Res.*, vol. 18, no. 177, pp. 1–86, 2018.
- [30] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.
- [31] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.
- [32] D. Wang, P. Cui, and W. Zhu, "Structural deep network embedding," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1225–1234.
- [33] M. Ou, P. Cui, J. Pei, Z. Zhang, and W. Zhu, "Asymmetric transitivity preserving graph embedding," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1105–1114.
- [34] K. Liang et al., "Knowledge graph contrastive learning based on relation-symmetrical structure," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 1, pp. 226–238, Jan. 2024.
- [35] K. Liang et al., "From concrete to abstract: Multi-view clustering on relational knowledge," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 10, pp. 9043–9060, Oct. 2025.
- [36] D. Zhu, Z. Zhang, P. Cui, and W. Zhu, "Robust graph convolutional networks against adversarial attacks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1399–1407.
- [37] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Social Netw.*, vol. 5, no. 2, pp. 109–137, 1983.
- [38] Y. Zhao, E. Levina, and J. Zhu, "Consistency of community detection in networks under degree-corrected stochastic block models," *Ann. Statist.*, vol. 40, no. 4, pp. 2266–2292, 2012.
- [39] Y. Liu et al., "A survey of deep graph clustering: Taxonomy, challenge, application, and open resource," 2022, *arXiv:2211.12875*.
- [40] S. Wang, J. Yang, J. Yao, Y. Bai, and W. Zhu, "An overview of advanced deep graph node clustering," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 1, pp. 1302–1314, Feb. 2024.
- [41] C. Holtz, P. Chen, Z. Wan, C.-K. Cheng, and G. Mishne, "Continuous partitioning for graph-based semi-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 5615–5645.



Shiyu Chen received the BE degree in computer science from Sun Yat-sen University, Guangzhou, China, in 2024, and the MS degree in data science from Johns Hopkins University, in 2025. His research interests include graph neural networks, random graph models, and statistical learning on graphs.



Cencheng Shen received the BS degree in quantitative finance from the National University of Singapore and the PhD degree in applied mathematics and statistics from Johns Hopkins University. He was an associate professor with the Department of Applied Economics and Statistics, University of Delaware. He is currently a principal research data scientist with Microsoft Research.



the Mathematical Institute for Data Science and the Human Language Technology Center of Excellence. His research interests are clustering algorithms, pattern classification, and data mining for high-dimensional and graph data.



Carey E. Priebe (Senior Member, IEEE) received the BS degree in mathematics from Purdue University, in 1984, the MS degree in computer science from San Diego State University, in 1988, and the PhD degree in information technology (computational statistics) from George Mason University, in 1993. From 1985 to 1994, he was a mathematician and scientist with U.S. Navy Research and Development Laboratory System. Since 1994, he has been a professor with the Department of Applied Mathematics and Statistics, Johns Hopkins University. His research interests include computational statistics, kernel and mixture estimates, statistical pattern recognition, model selection, and statistical inference for high-dimensional and graph data. He is also an elected member of the International Statistical Institute, fellow of the Institute of Mathematical Statistics, and fellow of the American Statistical Association.